R intro

ORF 245

02/02/2023

R Introduction

Download R at: https://cran.r-project.org/mirrors.html

Download R Studio at: https://www.rstudio.com/products/rstudio/download/

R-Markdown

Include the formula

$$
\begin{aligned}
x_n &= \sum_{i=1}^{n} i \\
&= \frac{n(n+1)}{2}
\end{aligned}
$$

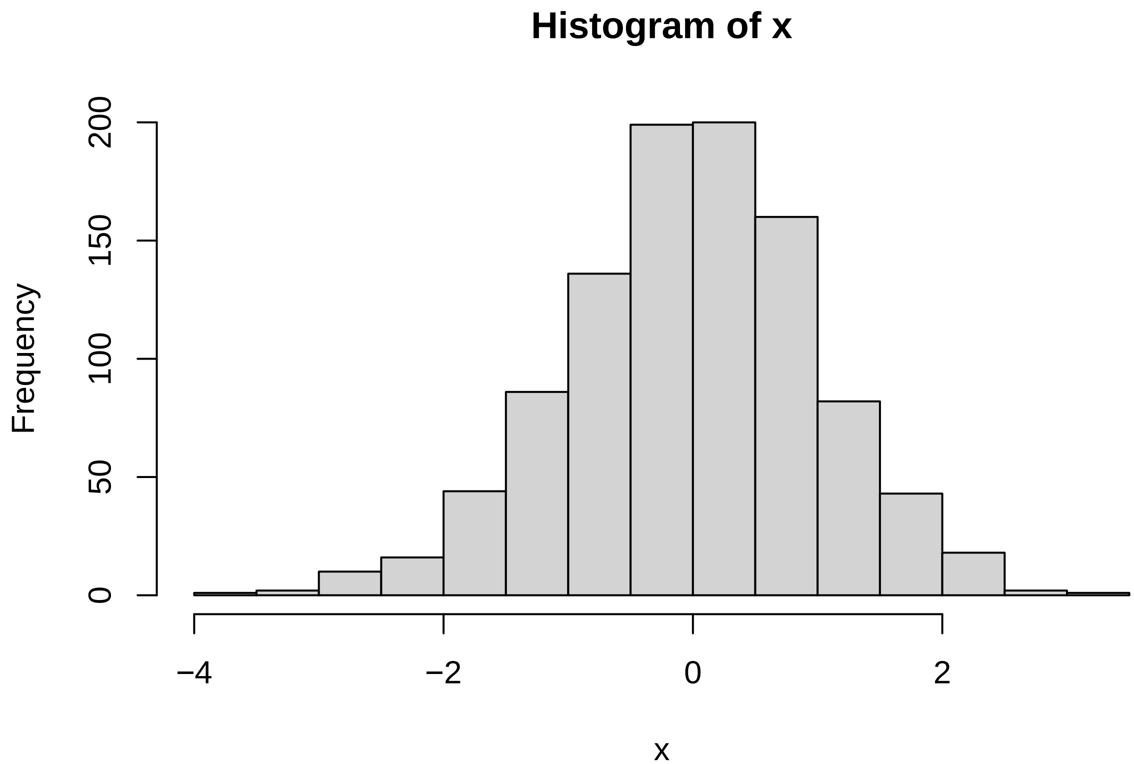
Include the code and the result

```
1 + 2
```

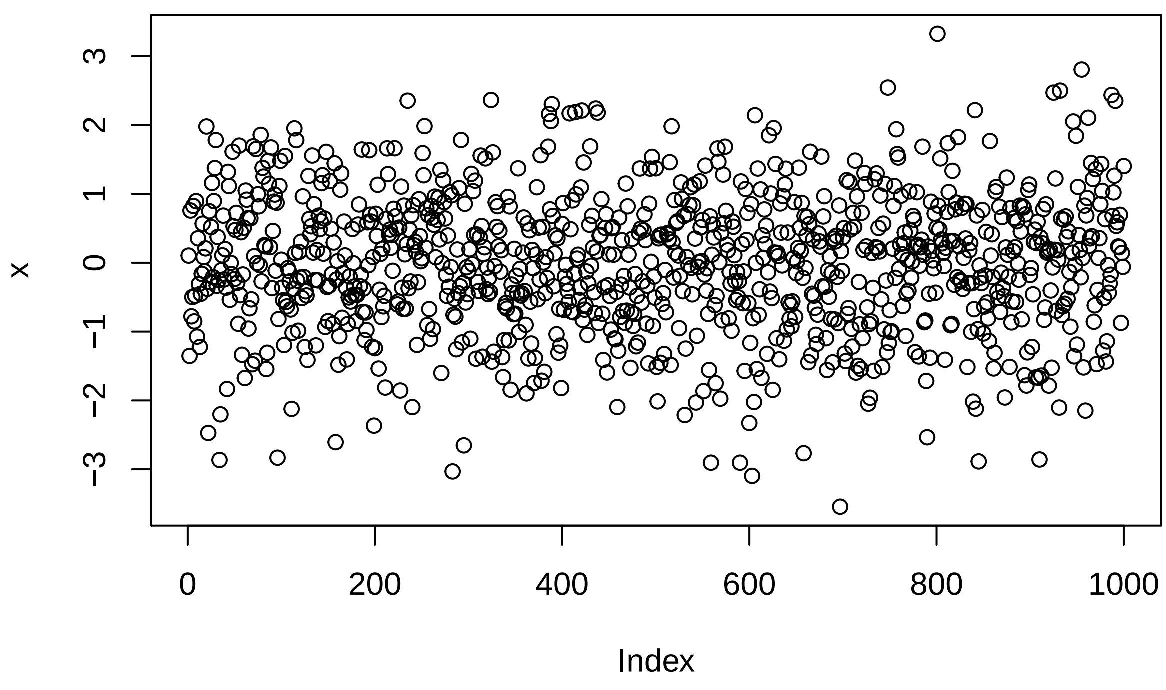
```
## [1] 3
```

Include the code and plotted figures

```
x <- rnorm(1000)
hist(x)
```




```
plot(x)
```



Basics

- R as a calculator
- vector
  - create a vector
  - vector operation

    - vector generator
- matrix
    - create a matrix
    - matrix operation
    - some functions
    - concatenate two matrices
- data.frame

```r
2 + 3
```

```
## [1] 5
```

```r
1.01 ^ 365
```

```
## [1] 37.78343
```

```r
# vector
x = c(1, 0, 3)
x = c(1, 0, 3, 0)
y = c(0, 2, 0, 4)
z = c(x, y)

x + y
```

```
## [1] 1 2 3 4
```

```r
x * y
```

```
## [1] 0 0 0 0
```

```r
# rep
a = rep(0, 100)
b = rep(c(1, 0, 3), 100)
c = rep(c(1, 0, 3), each=100)

# seq
a = seq(1, 10)
b = seq(1, 10, 0.5)
c = seq(2, 3, length.out=15)
```

```r
A = matrix(c(1, 2, 3, 4), nrow=2, ncol=2, byrow=TRUE)
B = matrix(c(0, 1, 1, 0), nrow=2, ncol=2, byrow=TRUE)

A + B
```

```
##      [,1] [,2]
## [1,]    1    3
## [2,]    4    4
```

```r
A * B
```

```
##      [,1] [,2]
## [1,]    0    2
## [2,]    3    0
```

```r
A %*% B
```

```
##      [,1] [,2]
## [1,]    2    1
## [2,]    4    3
```

```
t(A)
```

```
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
```

```
C = matrix(c(1, 2, 3, 4, 5, 6), nrow=2, ncol=3)
nrow(C)
```

```
## [1] 2
```

```
ncol(C)
```

```
## [1] 3
```

```
length(C)
```

```
## [1] 6
```

```
length(c)
```

```
## [1] 15
```

```
rbind(A, B)
```

```
##      [,1] [,2]
## [1,]    1    2
## [2,]    3    4
## [3,]    0    1
## [4,]    1    0
```

```
cbind(A, C)
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    2    1    3    5
## [2,]    3    4    2    4    6
```

R as a coding language

- function
  - print
  - summary statistics
- if-then-else
- for-loop and while loops
- logical statements

```
print('Hello World!')
```

```
## [1] "Hello World!"
```

```
cat('Hello World!')
```

```
## Hello World!
```

```
print(A)
```

```
##      [,1] [,2]
## [1,]    1    2
## [2,]    3    4
```

```r
x = seq(1, 10)
mean(x)
```

```
## [1] 5.5
```

```r
median(x)
```

```
## [1] 5.5
```

```r
min(x)
```

```
## [1] 1
```

```r
max(x)
```

```
## [1] 10
```

```r
sum(x)
```

```
## [1] 55
```

```r
for (i in 1:10){
  print(i)
}
```

```
## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
## [1] 6
## [1] 7
## [1] 8
## [1] 9
## [1] 10
```

```r
i = 1
while (i <= 10){
  print(i)
  i = i + 1
}
```

```
## [1] 1
## [1] 2
## [1] 3
## [1] 4
## [1] 5
## [1] 6
## [1] 7
## [1] 8
## [1] 9
## [1] 10
```

```r
x = 9
if (x <= 10){
  cat('x is less than or equal to 10')
}else{
  cat('x is greater than 10')
}
```

```
## x is less than or equal to 10
```

- data types
  - numeric value
  - int
  - complex
  - logical values: TRUE/FALSE
  - string/characters

```
(0+1i)^2
```

```
## [1] -1+0i
```

- logical statments
  - ==, <=, >=
  - ||, &&, &, |

```
a = 10
b = 10
c = 11
d = 10
if ((a == b) && (c <= d)){
  cat('YES!')
}
```

Data manipulation

- subsetting
- use vectorized operation

```
x = seq(0, 1, 0.1)
x[2]
```

```
## [1] 0.1
```

```
x[c(2, 5, 8)]
```

```
## [1] 0.1 0.4 0.7
```

```
x[2:8] # x[seq(2, 8)]
```

```
## [1] 0.1 0.2 0.3 0.4 0.5 0.6 0.7
```

```
x[x < 0.3]
```

```
## [1] 0.0 0.1 0.2
```

```
x < 0.3
```

```
##  [1]  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

```
A = matrix(seq(1, 100), nrow=10, ncol=10)
A[c(6, 7), c(2, 3, 5)]
```

```
##      [,1] [,2] [,3]
## [1,]   16   26   46
## [2,]   17   27   47
```

```
rowsum = function(A){
  ret = rep(0, nrow(A))
  for (i in 1:nrow(A)){
    ret[i] = sum(A[i,])
```

```r
  }
  return(ret)
}
A = matrix(c(1, 2, 3, 4), 2, 2, byrow=TRUE)
rowsum(A)
```

```
## [1] 3 7
```

```r
apply(A, 1, sum)
```

```
## [1] 3 7
```